

Chapter 4: Solving Problems That Have Dual Solution Characteristics

Nature Inspired Optimisation for Delivery Problems (2022)

Chapter Outline

- 1 4.1 Dealing with Conflicting Objectives
- 2 4.2 Pareto Dominance
- 3 4.3 NSGA-II: An Evolutionary Algorithm for Non-Dominated Fronts
- 4 4.4 Case Study: Routes Versus Customer Service
- 5 4.5 Results Summary
- 6 4.6 The Hypervolume Indicator
- 7 4.7 Conclusions

4.1 Why One Objective Is Often Not Enough I

Real-world delivery problems almost always involve tension between two or more goals that pull in opposite directions. A typical logistics manager cares about at least two things simultaneously: *cost* (keeping transport expenditure low) and *environmental impact* (reducing fuel consumption and emissions). Unfortunately, trying to minimise one of these two objectives usually makes the other one worse.

The Fundamental Tension

A solution that minimises cost typically achieves this by consolidating deliveries onto as few vehicles as possible, meaning each vehicle carries a heavy load and travels long, winding routes — which increases emissions and environmental impact. Conversely, minimising environmental impact may call for fewer kilometres driven, which can require more vehicles and thus higher operational costs.

4.1 Why One Objective Is Often Not Enough II

Before this chapter, the ACE Doughnuts logistics team used a single-objective approach: they would pick one criterion — say, total distance — as the fitness function, and the Evolutionary Algorithm (EA) would return a *single* best solution. However, the logistics manager found this approach frustrating. The returned solution did not always match their needs.

A Concrete Illustration

Suppose the single-objective EA returns a solution with 5 routes and a customer service metric of 5,466. This may be excellent for minimising routes, but the customer service level (lower is better in the book's formulation) may be far worse than what the manager actually needs. Meanwhile, optimising customer service alone may yield a solution with 32 routes — far too many vehicles for the available fleet.

4.1 Why One Objective Is Often Not Enough III

There are three different types of result that can emerge from a bi-objective optimisation. Understanding these three cases is essential before we go further.

- 1 **Find a solution that optimises c (cost, say) at the expense of e (environment).** This is the extreme point where c is as small as possible, with no regard for e .
- 2 **Find a solution that optimises e at the expense of c .** This is the opposite extreme.
- 3 **Find a range of trade-off solutions** that balance c and e in different proportions, allowing a human decision-maker to choose the trade-off that suits their current situation.

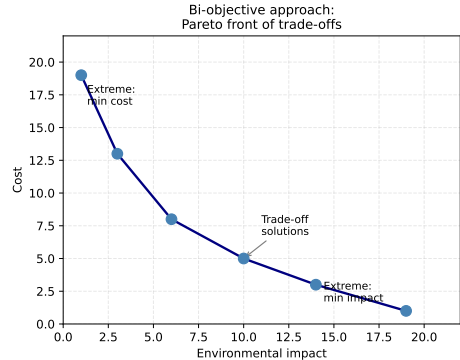
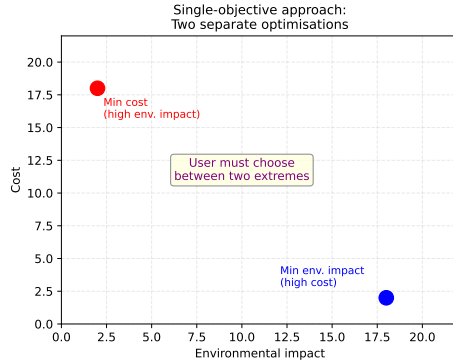
The third option is by far the most useful in practice, and it is the main subject of this chapter. We want to present the decision-maker with a *set* of solutions, not just one.

4.1 Why One Objective Is Often Not Enough IV

Key Insight

The goal of multi-objective optimisation is not to find *the* best solution, but to find the best *set* of solutions, each representing a different trade-off between the competing objectives. This empowers the human expert to make an informed final decision, taking into account factors that are difficult to quantify mathematically.

4.1 Why One Objective Is Often Not Enough V



On the left, single-objective optimisation yields only two extreme points — no middle ground is visible. On the right, bi-objective optimisation yields a full curve of trade-off solutions from which the manager can select.

4.2 Pareto Dominance — Formal Definition I

Named after the Italian economist **Vilfredo Pareto** (1848–1923), who observed that in economics, resource allocations can be improved without making anyone worse off until a certain boundary is reached — the “Pareto optimal” boundary. In optimisation, we adopt a similar idea.

We consider solutions that have two measurable characteristics, which we wish to *minimise*. Let us call these characteristics c and e .

Definition: Pareto Dominance (minimisation)

Solution x_1 **dominates** solution x_2 — written $x_1 \prec x_2$ — if and only if:

$$c(x_1) \leq c(x_2) \quad (\text{at least as good on } c) \tag{1}$$

$$e(x_1) \leq e(x_2) \quad (\text{at least as good on } e) \tag{2}$$

$$\text{and at least one inequality is strict.} \tag{3}$$

Symbols: $c(x_1)$ is the cost of solution x_1 ; $e(x_1)$ is its environmental impact. The symbol “ $\leq$ ” means “less than or equal to”. A strict inequality means “strictly less than”.

4.2 Pareto Dominance — Formal Definition II

In plain English: x_1 dominates x_2 when x_1 is *at least as good* on every objective, and *strictly better* on at least one. There is no reason to ever choose x_2 over x_1 .

4.2 Pareto Dominance — Formal Definition III

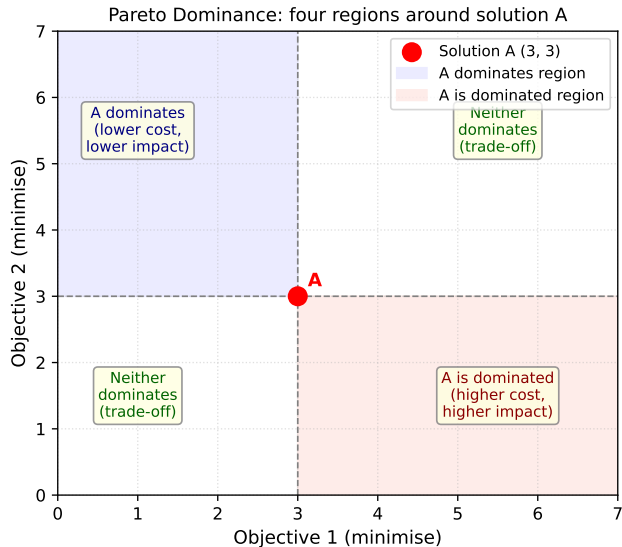
Worked Numerical Example

Consider three solutions with characteristics (c, e) :

Solution	c	e	Verdict
x_A	5	7	Non-dominated (our reference solution A)
x_1	12	8	Dominated by x_A (A has $c = 5 < 12$ and $e = 7 < 8$)
x_2	9	14	Dominated by x_A (A has $c = 5 < 9$ and $e = 7 < 14$)
x_3	14	15	Dominated by x_A (A has $c = 5 < 14$ and $e = 7 < 15$)

Solution $x_A = (5, 7)$ dominates all three because it is strictly better on both c and e . Since no other solution dominates x_A itself, x_A is called **non-dominated**.

4.2 Pareto Dominance — Formal Definition IV



The plot shows the four possible quadrants around any solution A. For minimisation:

4.2 Algorithm: Finding the Non-Dominated Front I

We now need a concrete algorithm that takes a population of solutions and extracts only those that are non-dominated. The logic is straightforward: for each solution in the population, check whether any *other* solution dominates it. If none does, it belongs to the non-dominated front.

Algorithm 11: FindNonDomFront

- ➊ Initialise an empty list `nonDom`.
- ➋ For each solution `possible` in the population:
 - ➊ Set `dominated = False`.
 - ➋ For each *other* solution in the population:
 - If that other solution dominates `possible`, set `dominated = True` and stop checking (break).
 - ➌ If `dominated` is still `False`, add `possible` to `nonDom`.
- ➌ Return `nonDom`.

The time complexity of this algorithm is $O(n^2)$ in the number of solutions n , because in the worst case every solution is checked against every other solution. For large populations this can be expensive, and

4.2 Algorithm: Finding the Non-Dominated Front II

faster algorithms exist (such as NSGA-II's fast non-dominated sort), but Algorithm 11 is easy to understand and implement.

4.2 Algorithm: Finding the Non-Dominated Front III

The Python implementation below translates the algorithm directly. The helper method `dominates(self, other)` returns `True` if the current solution dominates `other`, meaning it is at least as good on all objectives and strictly better on at least one.

Python: Finding the Non-Dominated Front

```
1  def dominates(self_vec, other_vec):
2      """Return True if self_vec dominates other_vec (minimisation).
3
4      self_vec and other_vec are lists of objective values,
5      e.g. [cost, env_impact].
6
7      'dominates' means: no worse on any objective,
8      strictly better on at least one.
9      """
10     at_least_as_good = all(s <= o for s, o in zip(self_vec, other_vec))
11     strictly_better   = any(s <  o for s, o in zip(self_vec, other_vec))
12     return at_least_as_good and strictly_better
13
14
15  def find_non_dom_front(population):
16      """Extract the non-dominated solutions from a population.
17
18      population: list of objects, each with a .get_vector() method
19                  returning a list of objective values.
20      Returns: list of non-dominated solution objects.
21      """
22     non_dom = []
23     for candidate in population:
24         dominated = False
25         for other in population:
26             if other is candidate:
27                 continue # do not compare a solution to itself
```

4.3 Adapting the Evolutionary Algorithm for Two Objectives I

In Chapter 3, the Evolutionary Algorithm (EA) used a single *fitness function* — a number indicating how good a solution was — to drive selection. Better fitness meant a higher chance of becoming a parent and producing offspring. With two objectives, there is no single number that captures “better”; instead, we have the concept of *dominance*.

Core Idea: Selection by Dominance

Instead of selecting the solution with the better fitness score, we select the solution that *dominates* the other. If neither dominates the other (they are incomparable), we keep one arbitrarily — both are equally acceptable trade-offs.

The modified selection mechanism is called **dominated tournament selection** (Algorithm 12 in the book). It picks two solutions at random from the population. If one dominates the other, the dominating solution is chosen as a parent. If neither dominates, either can be chosen.

4.3 Adapting the Evolutionary Algorithm for Two Objectives II

Algorithm 12: DominatedTournament

- 1 Pick solution x at random from the population.
- 2 Pick solution y at random from the population.
- 3 If x dominates y : return x .
- 4 Otherwise: return y .

Note that step 4 returns y regardless of whether y dominates x . This is a simplification: since neither dominates the other in the “otherwise” case (if y dominated x , step 3 would have caught it — actually the condition is asymmetric), both are acceptable choices.

4.3 Adapting the Evolutionary Algorithm for Two Objectives III

Important Distinction: Non-Dominated vs. Pareto Optimal

The EA returns a *non-dominated* front, but not necessarily the *Pareto optimal* front. The Pareto optimal front is the theoretically best set of non-dominated solutions, taking into account the *entire* solution space. Because the CVRP (Capacitated Vehicle Routing Problem) is NP-Complete (no polynomial-time exact algorithm is known), we cannot guarantee finding the Pareto optimal front in a reasonable time. The EA finds the best non-dominated front it can within the given computational budget.

4.3 Adapting the Evolutionary Algorithm for Two Objectives IV

The full evolutionary loop is modified in two important ways compared to the single-objective EA from Chapter 3:

- 1 **Batch child creation.** Instead of creating one child per iteration, we create a batch of `CHILDREN` children before updating the population. This amortises the cost of the computationally expensive non-dominated front extraction.
- 2 **Population pruning by dominance.** After adding all children to the population, we reduce the population to only its non-dominated members by calling `extractNonDom()`. This is what drives the algorithm toward the Pareto front over time.

The grand front — formed by combining the non-dominated fronts from multiple independent runs — is defined as:

4.3 Adapting the Evolutionary Algorithm for Two Objectives V

Grand Front Formula

$$\text{grandFront} = nd(p_1 \cup p_2 \cup \dots \cup p_{10})$$

where $nd(s)$ is a function that returns the set of non-dominated members of set s , and p_n (where the subscript n denotes the run number, from 1 to 10) is the non-dominated front returned by run n of the EA.

The union symbol $\cup$ (pronounced “union”) means we combine all solutions from all runs into one large pool, then extract the non-dominated ones. The grand front is at least as good as any individual run’s front, and typically better because different runs explore different parts of the objective space.

Python: Dominated Tournament and EA Main Loop

```
1  import random
2
3  def dominated_tournament(population):
4      """Select a solution using dominance-based tournament.
5
6      Picks two random solutions; returns the one that dominates
7      the other, or an arbitrary one if neither dominates.
8      """
9      x = random.choice(population)
10     y = random.choice(population)
11     if dominates(x.get_vector(), y.get_vector()):
12         return x
13     else:
14         return y      # y may dominate x, or neither may dominate
15
16
17 def bi_objective_ea(problem, evals_budget=10000,
18                     init_pop_size=100, children=10,
19                     xo_rate=0.7):
20     """Bi-objective Evolutionary Algorithm returning a non-dominated front.
21
22     problem          : the VRP problem instance
23     evals_budget     : total number of solution evaluations allowed
24     init_pop_size    : size of the initial random population
25     children         : number of children created per iteration (batch size)
26     xo_rate          : probability of crossover vs. mutation-only (0.7 = 70%)
27     """
```

4.4 The ACE Doughnuts Bi-Objective Problem I

ACE Doughnuts returns as our case study. Previously, the EA returned a single solution. The logistics manager has identified two characteristics that matter most when evaluating a delivery plan:

- 1 **Number of routes.** Fewer routes means fewer vehicles and drivers are needed, reducing operational costs. The manager wants to *minimise* this.
- 2 **Customer service metric.** This measures how well delivery times are distributed across customers — essentially a fairness and service quality score. A lower value (in the book's formulation) means better service because customers receive their orders closer to their preferred times. The manager also wants to *minimise* this.

These two objectives conflict with each other in a direct and obvious way. Reducing the number of routes means each vehicle serves more customers, which means longer and less tailored routes, which degrades customer service. Improving customer service means allocating each customer to a more specific route, which requires more routes overall.

4.4 The ACE Doughnuts Bi-Objective Problem II

Why Single-Objective Results Are Unsatisfactory

Table 4.1 in the book shows that optimising for routes alone can yield a customer service metric of 8,051.56 for problem A-n32-k5, while optimising for customer service alone yields 5,466.66 for the same problem. Neither extreme satisfies the manager, who wants a *choice* between these extremes.

A Sample of Single-Objective Results (Problem Set A)

Problem	Minimise Routes		Minimise Cust. Svc.	
	Routes	Cust. Svc.	Routes	Cust. Svc.
A-n32-k5	5	8051.56	5	5466.66
A-n33-k5	5	5231.27	6	3428.42
A-n33-k6	6	4748.40	6	2887.23
A-n34-k5	5	6351.24	6	4360.70
A-n36-k5	5	8597.31	6	5498.88

4.4 The ACE Doughnuts Bi-Objective Problem III

The table shows that the “best” number of routes depends on which objective is being optimised, and the two objectives genuinely conflict across all problem instances.

4.4 A New Encoding for Bi-Objective VRP I

To allow the EA to explore the trade-off between routes and customer service, the solution representation (encoding) must be modified. The original representation was simply a permutation of customer visits. The decoder then built routes by splitting the permutation at vehicle capacity boundaries. The new representation introduces a **“new route” flag** on each gene. Each gene now encodes *two* pieces of information:

- 1 The **customer ID** to be visited.
- 2 A **Boolean flag** (True/False) indicating whether this visit should be placed at the start of a *new route*, regardless of the current route's remaining capacity.

Example Genotype

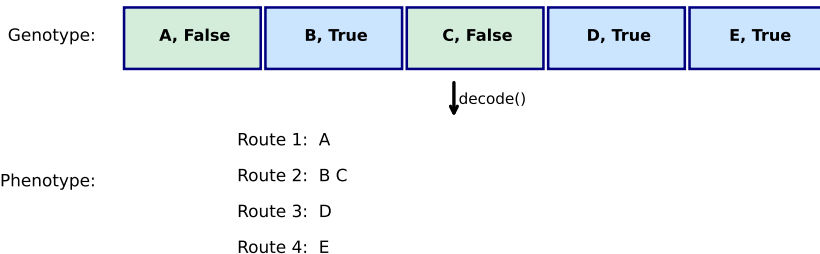
A, False	B, True	C, False	D, True	E, True
----------	---------	----------	---------	---------

With vehicle capacity 5 and demands $A = 2, B = 3, C = 2, D = 1, E = 2$.

4.4 A New Encoding for Bi-Objective VRP II

The decoder now processes the genotype gene by gene. Whenever it encounters a gene with the “new route” flag set to `True`, it immediately starts a new route — even if the current route still has remaining capacity.

Bi-objective genotype encoding: each gene = (customerID, newRoute flag)



4.4 A New Encoding for Bi-Objective VRP III

For the example genotype above, the decoder produces four routes:

Route	Customers
1	A
2	B C
3	D
4	E

Route 1 starts with customer A (no new-route flag yet, first gene). Route 2 starts because B has `newRoute = True`. Route 3 starts because D has `newRoute = True`. Route 4 starts because E has `newRoute = True`. Note that without any True flags, the original capacity-based decoder behaviour is preserved.

4.4 A New Encoding for Bi-Objective VRP IV

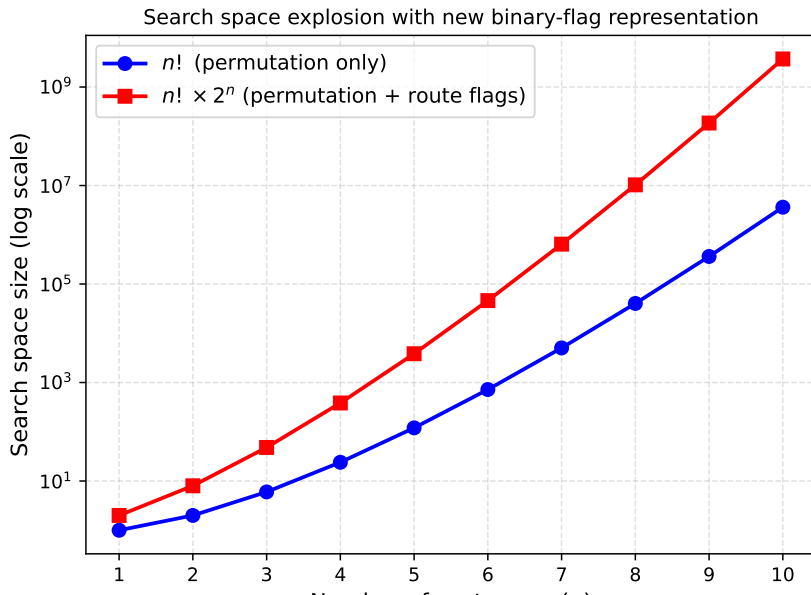
Search Space Explosion

The new representation increases the search space dramatically. Previously, with n customers, the search space was $n!$ permutations (the factorial of n , i.e. the number of ways to arrange n items in a sequence). Now, for each of those $n!$ permutations, there are 2^n possible binary strings (each of n genes can independently be True or False). The combined search space grows to $n! \times 2^n$.

n (customers)	$n!$	$n! \times 2^n$
1	1	2
2	2	8
3	6	48
4	24	384
5	120	3,840

Even for just 5 customers, the search space is 32 times larger than before. This makes exhaustive search impossible for real problem sizes, and reinforces why heuristic methods like the EA are necessary.

4.4 A New Encoding for Bi-Objective VRP V



Java: The Gene Class (Bi-Objective Encoding)

```
1  private class Gene {
2      /*
3       * Represents a single Gene within the bi-objective individual.
4       * visit      - the customer visit being made
5       * newRoute   - true if this visit should start a NEW route
6       */
7      private boolean newRoute = false;
8      private VRPVisit visit    = null;
9
10     public Gene(VRPVisit aVisit) {
11         this.visit      = aVisit;
12         this.newRoute   = false;    // default: no forced new route
13     }
14
15     public boolean newRoute() { return this.newRoute; }
16     public VRPVisit visit()   { return this.visit; }
17
18     public void setNewRoute(boolean val) { this.newRoute = val; }
19
20     // flipNewRoute() toggles the flag: True->False or False->True
21     public void flipNewRoute() {
22         this.newRoute = !this.newRoute;
23     }
24 }
```

Java: The Modified Decode Method

```
1 private void decode() {
2     if (phenotype == null) {
3         phenotype = new ArrayList<ArrayList<VRPVisit>>();
4         ArrayList<VRPVisit> newRoute = new ArrayList<VRPVisit>();
5
6         for (Gene g : genotype) {
7             VRPVisit v = g.visit();
8
9             if (g.newRoute()) {
10                 // Gene says "start a new route here" --
11                 // force a route break regardless of remaining capacity
12                 phenotype.add(newRoute);
13                 newRoute = new ArrayList<VRPVisit>();
14             }
15
16             if (v.getDemand() + routeDemand(newRoute)
17                 > problem.getCapacity()) {
18                 // Capacity exceeded -- must start a new route anyway
19                 phenotype.add(newRoute);
20                 newRoute = new ArrayList<VRPVisit>();
21             }
22
23             newRoute.add(v);
24         }
25         phenotype.add(newRoute);    // add the final route
26     }
27 }
```

Java: The NonDominatedPop Container and EA Loop

```
1  // NonDominatedPop: a population manager that maintains only  
2  // non-dominated solutions  
3  public NonDominatedPop extractNonDom() {  
4      NonDominatedPop result = new NonDominatedPop();  
5      for (Domination current : this) {           // for each solution  
6          boolean dominated = false;  
7          for (Domination member : this) {       // compare to every other  
8              if (member.dominates(current)) {   // if 'member' beats 'current'  
9                  dominated = true;  
10                 break;  
11             }  
12         }  
13         if (!dominated)  
14             result.add(current);               // only non-dominated survive  
15     }  
16     return result;  
17 }  
18  
19 // The main EA solve() loop  
20 public void solve() {  
21     population = new NonDominatedPop(INIT_POP_SIZE, super.theProblem);  
22     evalsBudget -= INIT_POP_SIZE;  
23  
24     while (evalsBudget > 0) {  
25         for (int count = 0; count < CHILDREN; count++) {  
26             BiObjectiveIndividual child;  
27             if (rnd.getRnd().nextDouble() < XO_RATE) {
```


4.4 Results of the Bi-Objective EA I

Tables 4.2, 4.3, and 4.4 in the book report the results of the bi-objective EA across three sets of CVRP benchmark instances. For each problem instance, the table reports:

- The **minimum and maximum number of routes** found in the non-dominated front (i.e. the extreme points on the “routes” dimension).
- The **minimum and maximum customer service metric** in the front.
- The **front size**: how many non-dominated solutions are in the grand front.

Selected Results for Problem Set A

Problem	Min Routes	Max Routes	Min Cust.Svc.	Max Cust.Svc.	Front Size
A-n32-k5	5	31	1870.38	6096.56	27
A-n33-k5	5	33	1272.05	4264.70	29
A-n39-k5	5	38	1806.09	6017.81	28
A-n44-k7	6	44	2019.97	7003.61	41
A-n80-k10	9	80	5576.9	23670.4	63

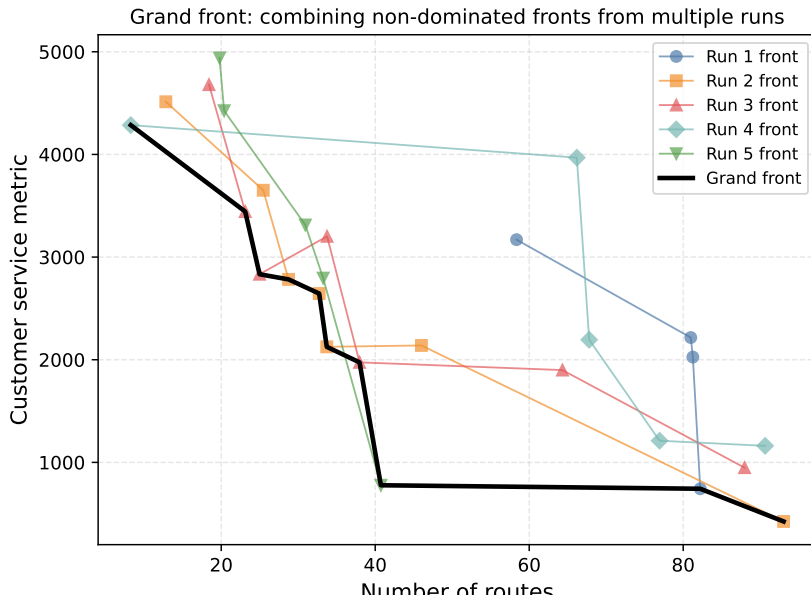
4.4 Results of the Bi-Objective EA II

The grand front for problem A-n32-k5 contains 27 non-dominated solutions, each offering a different trade-off between 5 routes (fewest vehicles but worst service) and 31 routes (most vehicles but best service, with customer service metric as low as 1,870).

4.4 Results of the Bi-Objective EA III

A visualisation of the non-dominated fronts across multiple runs, and the resulting grand front, illustrates how the EA explores the trade-off space.

4.4 Results of the Bi-Objective EA IV



4.5 Discussion of Results Across Problem Sets I

The bi-objective EA was tested on three sets of CVRP benchmark instances (sets A, B, and P), covering problems ranging from 16 customers (P-n16-k8) to 101 customers (P-n101-k4). The key observations from the results are:

- 1 **The lower bound on routes is usually equal to the minimum found by single-objective route minimisation.** This is expected: the extreme point of the Pareto front on the routes dimension corresponds to the best solution found by the single-objective EA.
- 2 **The upper bound on routes is approximately one route per customer** (when all new-route flags are set to True). This gives each customer their own dedicated vehicle for maximum service quality.
- 3 **Customer service improves monotonically as the number of routes increases.** More routes means shorter, more targeted delivery tours, which improves the customer service metric.
- 4 **The grand front adds value over individual runs.** For the B-n78-k10 instance, the grand front achieves a Hypervolume 2.2% above the average of the 10 individual run fronts, and 1.9% above the best individual run.

4.5 Discussion of Results Across Problem Sets II

Grand Front Table for A-n64-k9 (Selected Entries)

Routes	Cust.Svc.	Routes	Cust.Svc.	Routes	Cust.Svc.
9	14691.63	29	6429.22	52	4360.72
10	13448.21	30	6101.75	53	4152.55
11	13036.44	32	5927.05	56	4096.42
12	11934.61	33	5728.70	57	4038.34
15	10960.03	36	5474.53	60	3902.84
20	8633.49	40	5071.66	64	3816.04

This table provides the logistics manager with a concrete menu of choices. If the manager has a fleet of 20 vehicles, they can see exactly what customer service level (8,633.49) is achievable. If they can secure two additional vehicles (22 routes), the customer service improves to around 8,259.

4.5 Discussion of Results Across Problem Sets III

Practical Value

The grand front enables scenario analysis. The planner can ask: “What is the benefit of increasing our fleet from 20 to 25 vehicles?” The answer is read directly from the table: customer service improves from 8,633 to approximately 7,835 — a 9.3% improvement for a 25% fleet increase. This kind of cost-benefit analysis is impossible with a single-objective approach.

4.6 Comparing Two Non-Dominated Fronts I

Once we have non-dominated fronts, a natural question arises: how do we compare the quality of two different fronts? For example, if we run two different algorithms and each produces a Pareto front, which algorithm produced the better front?

Directly comparing point by point is difficult because the two fronts may have different numbers of points at different positions. We need a single scalar (one number) that summarises the quality of an entire front.

4.6 Comparing Two Non-Dominated Fronts II

The Hypervolume Indicator (Definition)

The **Hypervolume Indicator** (also called the S-metric or HV) measures the n -dimensional volume (in 2D: the area; in 3D: the volume) of the *objective space* that is dominated by the non-dominated front and bounded by a reference point called the **nadir point**.

Formally, for a front F and nadir point z_{nadir} :

$$\text{HV}(F, z_{\text{nadir}}) = \lambda \left(\bigcup_{x \in F} \{a : x \prec a \prec z_{\text{nadir}}\} \right)$$

where $\lambda(\cdot)$ denotes the n -dimensional Lebesgue measure (i.e. area in 2D, volume in 3D), $x \prec a$ means x dominates point a , and z_{nadir} is the nadir point.

4.6 Comparing Two Non-Dominated Fronts III

Symbol Guide for the Hypervolume Formula

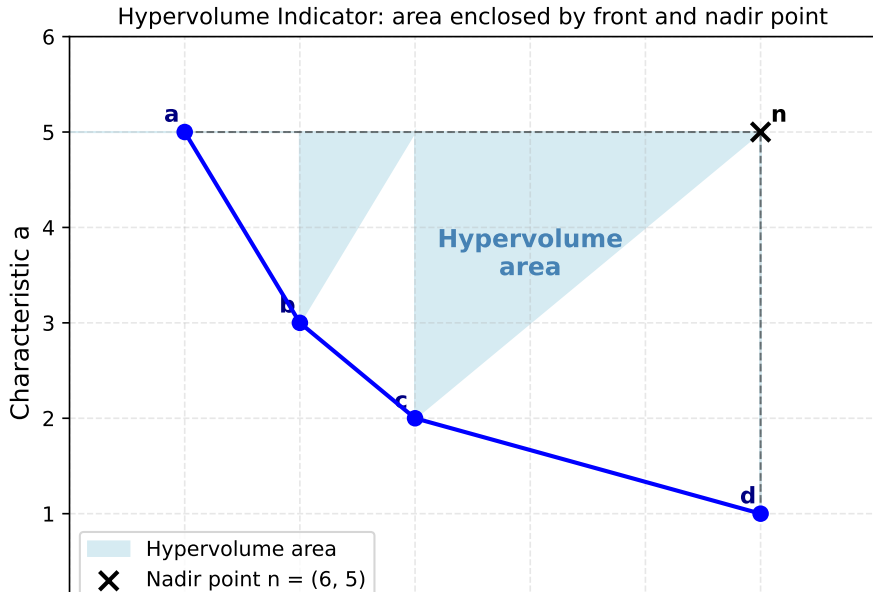
- F — the set of non-dominated solutions (the front).
- z_{nadir} (nadir point) — a reference point whose coordinates are the *worst* objective values observed across all solutions being compared. In 2D, if the worst cost is 5 and the worst environmental impact is 6, then $z_{\text{nadir}} = (5, 6)$.
- $\lambda(\cdot)$ — the Lebesgue measure, meaning area in 2D or volume in 3D. It is the standard measure of “size” of a region.
- $\bigcup$ (big union, “union of all sets”) — the union over all solutions x in the front F .
- $x \prec a$ — solution x dominates point a , meaning all coordinates of x are less than or equal to the corresponding coordinates of a .

4.6 Comparing Two Non-Dominated Fronts IV

How to Read the Hypervolume Formula

The formula builds a region by taking every point in the objective space that is dominated by at least one solution in F *and* also dominated by the nadir point. The hypervolume is the total area (or volume) of this region. A larger hypervolume means the front extends further into the objective space — it contains solutions that are better across a wider range of trade-offs. Think of it as: the more area your front “covers”, the better your algorithm is performing.

4.6 Comparing Two Non-Dominated Fronts V



Python: Computing the Hypervolume in 2D

```
1 def hypervolume_2d(front, nadir):
2     """Compute the exact hypervolume for a 2D non-dominated front.
3
4     front : list of (obj1, obj2) tuples, sorted by obj1 ascending.
5             Both objectives are to be minimised.
6     nadir : (nadir_obj1, nadir_obj2) reference point -- the worst
7             values on each objective, used as the upper-right corner.
8
9     Returns the hypervolume (area) as a float.
10    """
11    # Sort front by first objective ascending (left to right)
12    front = sorted(front, key=lambda p: p[0])
13
14    total_area = 0.0
15    prev_x = 0.0    # start from x = 0 (or the leftmost bound)
16
17    for (x, y) in front:
18        # Width of the current vertical strip
19        width = x - prev_x
20        # Height: from nadir_y down to the front's y value
21        height = nadir[1] - y
22
23        if height > 0:    # only count if the strip has positive height
24            total_area += width * height
25
26        prev_x = x    # advance the left boundary to the current point
27
```

4.7 Conclusions and Key Takeaways I

This chapter has introduced the fundamental ideas of multi-objective optimisation as applied to vehicle routing problems. The key concepts are:

- 1 **Conflicting objectives are the norm, not the exception.** Real delivery planning almost always involves at least two goals that trade off against each other. Trying to solve this with a single objective either ignores one concern entirely or requires the user to pre-assign weights that are difficult to determine.
- 2 **Pareto dominance provides a natural way to compare solutions without needing weights.** Solution x_1 dominates x_2 if it is at least as good on every objective and strictly better on at least one. Non-dominated solutions cannot be improved on any objective without degrading another.
- 3 **The Evolutionary Algorithm adapts naturally to multi-objective optimisation** by replacing fitness-based selection with dominance-based selection. The rest of the EA infrastructure (crossover, mutation, encoding) requires only modest modifications.
- 4 **The Hypervolume Indicator provides a single-number quality measure for Pareto fronts,** enabling fair comparison between algorithms or between runs of the same algorithm. A higher hypervolume means a better-quality front.

4.7 Conclusions and Key Takeaways II

- 5 **The Grand Front, formed by combining multiple EA runs, consistently outperforms any single run** as measured by Hypervolume. This is because different runs explore different parts of the objective space, and their union covers more ground.

4.7 Conclusions and Key Takeaways III

The Broader Philosophical Point

The approach in this chapter represents a shift in thinking: instead of asking “what is the best solution?”, we ask “what is the best *set* of solutions?”. The optimisation algorithm’s job is not to make the final decision for the logistics manager, but to filter the enormous solution space down to a small, high-quality set of trade-offs that a human expert can evaluate.

This perspective — multi-objective optimisation as *decision support* rather than *decision making* — is increasingly important as logistics problems grow in scale and complexity.

4.7 Conclusions and Key Takeaways IV

Limitations and Future Directions

- The bi-objective representation increases the search space by a factor of 2^n compared to the single-objective representation. For large n , this can slow convergence.
- The distance objective was deliberately excluded in this chapter. In the next chapter, we will see how to extend these ideas to *three or more* objectives simultaneously.
- The non-dominated front found by the EA is not guaranteed to be the true Pareto optimal front; it is merely the best approximation the algorithm can find within the given evaluation budget.
- Different users may have different preferences over the trade-off space. Techniques such as preference articulation, reference point methods, and interactive multi-objective optimisation can help guide the search toward regions of the front that a particular user cares about most.

4.7 Conclusions and Key Takeaways V

Chapter Summary: Concept Map

Concept	What it is	Why it matters
Conflicting objectives	Two goals that cannot be simultaneously minimised	Motivates multi-objective approach
Pareto dominance	$x_1 \prec x_2$ if better or equal on all objectives, strictly better on one	Defines “better” without weights
Non-dominated front	Set of solutions no one dominates	The useful output for decision-makers
Domination tournament	Selection via Pareto dominance	Drives EA toward non-dominated solutions
Bi-objective encoding	Customer + newRoute flag per gene	Allows EA to explore routes vs. service trade-off
Grand front	Union of fronts from multiple EA runs	Better coverage of the Pareto front
Hypervolume	Area dominated by the front	Single-number quality score for a front

- Guerreiro, A.P., Fonseca, C.M., Paquete, L. (2020). *The Hypervolume Indicator: Problems and Algorithms*. arXiv:2005.00515.
- Fonseca, C.M., Lopez-Ibanez, M., Paquete, L., Guerreiro, A.P. (2010). Computation of the Hypervolume Indicator. <http://lopez-ibanez.eu/hypervolume>.
- Augerat, P. (1995). *Approche polyédrale du problème de tournées de véhicules (Polyhedral approach of the vehicle routing problem)*. Ph.D. Thesis, Grenoble Institute of Technology.
- Vilfredo Pareto (2021). Encyclopædia Britannica. <https://www.britannica.com/biography/Vilfredo-Pareto>.
- Deb, K., Pratap, A., Agarwal, S., Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197.